Algorithmic Roster Construction: Strategic Talent Acquisition under Competitive Scarcity

A Fantasy Baseball Testbed for Optimization and Real-Time Heuristics

OR114-2 Final Project, Group 4

B13705051 周孟承　　B13902103 鄭宇宏　　B13303153 詹舒宇　　B11705039 李盈盈

Spring 2026

Abstract

In this report, we study a **sequential resource allocation problem under competitive scarcity**. We utilize **Fantasy Baseball as a data-rich testbed** due to its complex roster constraints and high-frequency market dynamics. For modern sports organizations, identifying talent is only one part of the challenge; the true competitive edge lies in the **strategy of draft execution**. While traditional Integer Programming (IP) provides a mathematical benchmark, it fails to scale to massive player pools under real-time pressure. We propose the *Opportunity Cost Greedy (OCG)* algorithm, which captures the **cost-of-delay** by forecasting future asset availability. Our results across synthetic and real-world 2026 data show that OCG achieves near-optimal performance in seconds, effectively functioning as a scalable decision-support framework.

1 Introduction: The General Manager's Dilemma

The draft is a critical portfolio optimization puzzle for modern sports roster construction. In an era where scouting analytics and player evaluations have largely converged, the competitive edge has shifted from identifying talent to the **strategy of execution**. This project models the draft process as a **sequential resource acquisition problem**. We use Fantasy Baseball not as the final application itself, but as a high-fidelity competitive testbed to study how decision-makers can maximize objective utility under strict structural constraints and intense market competition.

In this environment, the manager must answer a fundamental question: *Which asset must I secure now because market timing and positional scarcity will cause equivalent options to disappear by my next turn?* This is the "Timing Paradox"—balancing immediate projected value with long-term roster integrity. We aim to build a framework that provides a deployable balance between theoretical optimality and real-time execution speed.

While the actual MLB draft uses a fixed order based on reverse standings, we abstract our environment into a **Snake Draft**. This format serves as a useful test environment for a highly competitive, fair, and strategically demanding sequence of decisions. In a snake draft, a manager owns a fixed sequence of picks, and the player pool is continually depleted by opponents. The practical decision is not simply selecting the player with the highest isolated score (the naive "Best Player Available" approach), but deciding how to trade off immediate value, positional scarcity, and future availability.

This decision is inherently interdependent. A high-projection player might be suboptimal if he occupies a position where supply is abundant, whereas a lower-projection player might be highly

valuable if he fills a scarce roster slot. To solve this, we translate the draft into a mathematical model.

Initially, an Average Draft Position (ADP)-aware **Integer Programming (IP)** model is formulated. While IP provides a flawless mathematical optimum, it encounters a severe roadblock in reality. Because baseball players do not need to actively register for the draft (anyone meeting age/school requirements is eligible), the actual draft pool scales to tens of thousands of players. When the problem scale rises, the IP algorithm exceeds reasonable time limits or crashes completely due to exponential complexity.

To solve this, we introduce a self-designed algorithm: **Opportunity Cost Greedy (OCG)**. This heuristic explicitly calculates the mathematical cost of waiting by peeking into the future availability of players. The overarching narrative of this report is to prove to the front office that OCG can achieve incredibly tight optimality gaps (retaining maximum roster value) while executing in mere seconds, even when the player pool scales to massive extremes.

2 Problem Description

We consider a snake draft environment where a manager operates a predetermined set of picks. At each owned pick, the manager must choose exactly one available player from the remaining pool. Every player in the pool possesses three attributes:

1. **Projected Value**: The expected statistical output (points) for the season.
2. **Eligible Positions**: The real-life positions the player can occupy.
3. **Average Draft Position (ADP)**: A market consensus of when the player is expected to be drafted by opponents.

The objective is to maximize the sum of projected points for the starting roster. The drafted players must perfectly fulfill strict positional requirements. The base roster structure used in our study demands 16 active slots, as detailed in Table 1. (Note: The Utility (`Util`) slot is hitter-only, meaning pitchers cannot be assigned there).

Table 1: Base fantasy baseball roster requirements.

Position	C	1B	2B	3B	SS	OF	Util	SP	RP
Required Slots	1	1	1	1	1	3	1	5	2

A player can only be drafted if he is still available in the market. We treat ADP as a proxy for market depletion. If the manager's current draft pick occurs much later than a player's ADP, that player is considered successfully drafted by an opponent and is unavailable. By integrating ADP market constraints and positional flexibility into the sequence of decisions, draft planning is transformed from a subjective ranking exercise into a strict operations research problem.

3 Model Formulation

We first formulate the exact ADP-aware Integer Programming (IP) model to serve as our absolute benchmark for roster quality.

Let I be the set of players, P the set of roster positions, and K the set of owned draft picks. Each player $i \in I$ has projected points v_i, market expectation adp_i, and eligible positions $E_i \subseteq P$. Each position $p \in P$ requires r_p players. Let S_k be the overall draft number for the manager's k-th pick.

Decision Variables:

- $y_i \in \{0, 1\}$: 1 if player i is drafted, 0 otherwise.
- $x_{ip} \in \{0, 1\}$: 1 if player i is assigned to position p, 0 otherwise.
- $z_{ik} \in \{0, 1\}$: 1 if player i is selected precisely at owned pick k, 0 otherwise.

Objective Function: Maximize total roster points.

$$\max \sum_{i \in I} v_i y_i$$

Constraints:

$$\sum_{i \in I} z_{ik} = 1, \qquad \forall k \in K \tag{1}$$

$$\sum_{k \in K} z_{ik} = y_i, \qquad \forall i \in I \tag{2}$$

$$\sum_{p \in P} x_{ip} = y_i, \qquad \forall i \in I \tag{3}$$

$$\sum_{i \in I} x_{ip} = r_p, \qquad \forall p \in P \tag{4}$$

$$z_{ik} = 0 \quad \text{if } S_k > \text{adp}_i + \delta, \qquad \forall i \in I, k \in K \tag{5}$$

$$y_i, x_{ip}, z_{ik} \in \{0, 1\} \qquad \forall i, p, k \tag{6}$$

Constraint (1) enforces exactly one selection per pick. (2) and (3) ensure logical consistency between picking, drafting, and assigning. (4) dictates the roster shape. Constraint (5) is the vital **Market Availability Constraint**: a player cannot be drafted if the current pick S_k is later than his expected depletion point (ADP + a tolerance buffer δ).

4 Algorithms

While the IP provides mathematical perfection, solving it is NP-Hard. For massive player pools, the branch-and-bound tree explodes. Therefore, we developed two heuristic algorithms using Priority Queues (Heaps) to achieve extreme computational efficiency.

4.1 Direct Greedy (Baseline)

The Direct Greedy (DG) algorithm serves as our baseline. It represents the standard "fill the biggest need with the best available player" mentality.

At each pick, DG scans all open positions, identifies the position with the highest scarcity (Remaining Need / Available Players), and selects the player with the highest projected points for that position.

Algorithm 1 Direct Greedy (DG)

Initialize Max-Heaps for each position $p \in P$ based on v_i.
Sort all players by $(\text{adp}_i + \delta)$ to create an `expiration_order`.
for each owned pick $k \in K$ **do**
    Pop expired players from heaps where $\text{adp}_i + \delta < S_k$
    best_position $\leftarrow$ null, max_scarcity $\leftarrow -\infty$
    for each position p with $r_p > 0$ **do**
        Clean heap (remove drafted/expired players lazily)
        scarcity $\leftarrow \frac{\text{Remaining Need for } p}{\text{Active Eligible Players for } p}$
        if scarcity > max_scarcity (tie-break by top player's v_i) **then**
            best_position $\leftarrow p$
            max_scarcity $\leftarrow$ scarcity
        end if
    end for
    Draft top player from best_position heap and update remaining needs.
end for

4.2 Opportunity Cost Greedy (Proposed Method)

Opportunity Cost Greedy (OCG) is our flagship algorithm. Instead of only looking at the *current* turn, OCG looks ahead to the *next* owned pick. It calculates the **Opportunity Cost** of waiting for a position by subtracting the points of the best player available *next round* from the best player available now.

Algorithm 2 Opportunity Cost Greedy (OCG)

Initialize two sets of Max-Heaps for each position p: `current_heaps` and `future_heaps`.
for each owned pick $k \in K$ (let next pick be k_{next}) **do**
    Expire players in `current_heaps` based on S_k
    Expire players in `future_heaps` based on $S_{k_{next}}$
    best_choice $\leftarrow$ null, max_priority $\leftarrow (-\infty, -\infty, -\infty)$
    for each position p with $r_p > 0$ **do**
        $v_{cur} \leftarrow$ points of top player in `current_heaps`$[p]$
        $v_{fut} \leftarrow$ points of top player in `future_heaps`$[p]$
        $score \leftarrow v_{cur} - v_{fut}$ ▷ Delay Cost Calculation
        forced $\leftarrow$ True if (Active Players $\leq$ Need)
        priority $\leftarrow$ (forced, $score, v_{cur}$)
        if priority > max_priority **then**
            max_priority $\leftarrow$ priority
            best_choice $\leftarrow$ (Top Player, p)
        end if
    end for
    Draft player from best_choice and update remaining needs.
end for

4.3 Time Complexity Analysis

Let n be the total number of players in the draft pool, r be the number of roster slots (total picks per manager), and p be the number of distinct positions.

- **Initialization**: Building the initial max-heaps and sorting the expiration list takes $O(n \log n)$.
- **Drafting Loop**: Checking the top of a heap is $O(1)$. For each of the r picks, the algorithm scans p positions.

- **Heap Cleaning (Lazy Deletion)**: Expiring or removing drafted players is done by popping invalid elements at the top of the heap. Across the *entire draft process*, a player is popped from a position heap at most once. Therefore, the total time spent cleaning heaps is bounded by $O(n \log n)$.

Total Time Complexity for both DG and OCG is $O(n \log n + r \cdot p)$. Because r and p are tiny constants relative to n, the algorithm runs in near-linear time relative to sorting the player pool. This mathematically guarantees extreme scalability compared to the exponential worst-case nature of IP.

5 Data Collection and Generation

To evaluate our algorithms, we utilize both real-world data and a highly customizable **Synthetic Data Generator**.

5.1 Real Data Collection

Real data validation serves as our first step of validation. We synthesize our dataset from FantasyPros, utilizing **projected points as a player utility proxy** (v_i) and **Average Draft Position (ADP) as a market availability window proxy**. Figures 1 and 2 display examples of the underlying data structure.

PLAYER	AB	R	HR	RBI	SB	AVG	OBP	H	2B	3B	BB	SO	SLG	OPS	ROST%
Shohei Ohtani (LAD - SP,DH)	572	124	47	111	24	.283	.382	162	28	5	92	164	.598	.980	100%
Aaron Judge (NYY - LF,CF,RF,DH)	520	110	46	114	9	.292	.417	152	26	1	112	160	.609	1.026	99%
Bobby Witt Jr. (KC - SS)	603	101	29	93	34	.292	.345	176	38	6	49	114	.518	.863	99%

Figure 1: Example of 2026 player projected batting and pitching statistics.

Import a Team

2026

Rank	Player (Team)	Yahoo	CBS	RTS	NFBC	FT	ESPN	AVG
1	Shohei Ohtani (LAD - SP,DH)		1	2	1	1	1	1.0
2	Aaron Judge (NYY - LF,CF,RF,DH)	1	2	1	2	2	2	1.8
3	Shohei Ohtani (Batter) (LAD - DH)	2						2.0
4	Bobby Witt Jr. (KC - SS)	3	3	4	3	4	4	3.6
5	Juan Soto (NYM - LF,RF,DH)	4	4	3	4	3	3	3.6

Figure 2: Example of player Average Draft Position (ADP).

Player values are calculated using two distinct standard scoring benchmarks: Yahoo (which focuses on traditional counting stats) and FanGraphs (which incorporates more advanced player metrics based on linear weights). The detailed scoring weights for both batters and pitchers are outlined in Table 2.

Table 2: Scoring weights for Yahoo and FanGraphs formats.

Batters			Pitchers		
Metric	**Yahoo**	**FanGraphs**	**Metric**	**Yahoo**	**FanGraphs**
1B (or H)	2.6	5.6	IP	3.0	7.4
2B	5.2	2.9	K	3.0	2.0
3B	7.8	5.7	W	8.0	–
HR	10.4	9.4	SV	8.0	5.0
R	1.9	–	ER	-3.0	–
RBI	1.9	–	BB	-1.3	-3.0
BB	2.6	3.0	H	-1.3	-2.6
SB	4.2	1.9	HBP	-1.3	-3.0
HBP	0 (2.6)	0 (3.0)	HR	–	-12.3
AB	–	-1.0	HLD	–	4.0
CS	–	0 (-2.8)			

Slight modifications were made to the original projected stats to align with these scoring categories. For example, since the projected stats lack Hit By Pitch (HBP) and Caught Stealing (CS) for batters, we effectively set their scoring weights to zero (original standard weights are shown in parentheses in the table).

Additionally, across all experiments (both real and synthetic data), the manager's initial draft position is standardized to the middle of the draft order (e.g., the 6th pick in a 12-team league, or the 12th pick in a 24-team league) to maintain a consistent evaluation baseline.

5.2 Synthetic Data Generation

While real data confirms practical viability, the most important and further validation is performed on synthetic data. Synthetic data is indispensable as it allows us to mathematically isolate specific operational variables (Factors)—such as talent distribution, positional versatility, and market supply —and stress-test the algorithms under controlled extreme conditions.

The total number of generated players in the synthetic environment is defined dynamically based on the league size (T), base roster slots ($R = 16$), roster scale multiplier (S), and player-demand ratio (D):

$$N_{\text{players}} = S \times 16 \times T \times D$$

Our experimental framework generates synthetic profiles encompassing four primary operational factors:

5.3 Factor A: Points Distribution (v_i)

We designed three points distributions to simulate different talent pools, clipping all generated values to a realistic range of $[10.0, 800.0]$ according to the real data we collected above:

- **Normal (Baseline)**: Values follow a normal distribution $v_i \sim \mathcal{N}(400, 120^2)$. This represents an average draft class where the majority of players are solid, mid-tier contributors.
- **Uniform**: Values are uniformly distributed $v_i \sim \mathcal{U}(10, 800)$. This flattens the talent curve, offering many viable replacement options and minimizing scarcity.
- **High-Low (Star-Heavy)**: Simulates a highly skewed draft class. Only 10% of players are classified as elites, with values drawn from $\mathcal{U}(500, 800)$. The remaining 90% fall into $\mathcal{U}(10, 500)$. In this scenario, missing out on top-tier talent is severely punished in the objective function.

5.4 Factor B: Position Eligibility Distribution (E_i)

We control how versatile the player pool is using four distinct generation logics:

- **Roster-Ratio (Baseline)**: Players are assigned exactly one position, but the probability of generating a specific position aligns precisely with roster demand (e.g., more OF and SP players are generated).
- **Uniform-by-Type**: Randomly designates a player as a hitter or pitcher, then uniformly assigns a position pattern (including dual-eligibility like `1B;3B` or `SP;RP`).
- **Point-Flexible**: The most forgiving environment. The generator ranks all players by points. Players in the top 20% (elite) have an 80% chance of gaining multi-position eligibility (up to 3 positions). This mimics real life where superstars often possess high versatility.
- **Single-Position**: Every player possesses exactly one eligible position. This eliminates all flexibility, representing the ultimate test of scarcity and making early greedy mistakes almost impossible to repair.

5.5 Factor C: Market Conditions and ADP Uncertainty (σ_{adp})

To generate ADP, we first sort all players by their true projected points in descending order to establish a "True Rank". We then simulate market noise by adding a Gaussian error term:

$$\text{ADP}_i = \text{TrueRank}_i + \mathcal{N}(0, \sigma^2_{\text{adp}})$$

- $\sigma_{\text{adp}} = 0$ represents a perfectly efficient market where opponents draft strictly by objective value.
- Higher σ_{adp} introduces market chaos. Opponents may make unpredictable "reaches" or leave bargains on the board. This tests how robust our algorithms are when future availability is uncertain.

5.6 Factor D: Player Demand Ratio (D)

To test the algorithms under varying levels of scarcity, we manipulate the Player Demand Ratio (D). This factor controls the supply-and-demand economics of the draft pool by dictating the ratio of "Available Players" versus "Total League Demand". Let T be the number of teams and $R = 16$ be the base roster slots. The total number of generated players is strictly governed by $N_{\text{players}} = R \times T \times D$.

- $D = 1$ **(Demand: High)**: The league will draft almost every available player. One greedy mistake means replacing a starter with absolute zero value.
- $D = 3$ **(Baseline Supply)**: Represents standard fantasy draft conditions with a typical buffer of undrafted free agents.
- $D = 10$ **(High Abundance)**: The player pool is a massive ocean of options.

5.7 Scenario Matrix

We developed 12 distinct scenarios to isolate the impact of each factor. As shown in Table 3, **Scenario S1** serves as the baseline for all comparative analyses. The remaining scenarios follow the factor order introduced above: projected points distribution, position eligibility distribution, market conditions, and player demand ratio. For every scenario, we generate **10 random instances** to calculate the mean (μ) and standard deviation (σ) of the performance metrics.

Table 3: Experimental Scenario Matrix for Draft Optimization

ID	Main Factor	Demand Ratio (D)	Points Distribution (v_i)	Position Eligibility (E_i)	ADP Uncertainty (δ, σ)
S1	**Baseline**	3	Normal	Roster-Ratio	(0, 30)
S2	Points: Uniform	3	**Uniform**	Roster-Ratio	(0, 30)
S3	Points: Star-Heavy	3	**High-Low**	Roster-Ratio	(0, 30)
S4	Pos: Uniform-by-Type	3	Normal	**Uniform-by-Type**	(0, 30)
S5	Pos: Point-Flexible	3	Normal	**Point-Flexible**	(0, 30)
S6	Pos: Single-Position	3	Normal	**Single-Position**	(0, 30)
S7	Market: Efficient	3	Normal	Roster-Ratio	**(0, 0)**
S8	Market: Mild Noise	3	Normal	Roster-Ratio	**(0, 60)**
S9	Market: Chaotic	3	Normal	Roster-Ratio	**(+5, 30)**
S10	Market: Chaotic	3	Normal	Roster-Ratio	**(+10, 30)**
S11	Demand: High	**1**	Normal	Roster-Ratio	(0, 30)
S12	Demand: Low	**10**	Normal	Roster-Ratio	(0, 30)

5.8 Stress Testing Design

To evaluate the absolute computational limits of our algorithms relative to the IP benchmark, we design a separate **Large-Scale Stress Test** outside of the standard factor matrix. Instead of sweeping isolated market parameters, this test scales the raw problem dimensions—drastically increasing both the available player pool and the roster constraints. The primary objective is to simulate massive, unconstrained draft databases to observe the breaking point of exact optimization (IP) and to validate the asymptotic time efficiency of the OCG heuristic under massive scale.

6 Performance Evaluation

Algorithms are evaluated using two primary metrics across 10 random seeds for each scenario:

- **Optimal Gap Ratio (%)**: The distance from the IP benchmark, defined as $\frac{Z_{\text{IP}} - Z_{\text{Heuristic}}}{Z_{\text{IP}}} \times 100\%$. A lower value indicates higher accuracy.
- **Improvement (%)**: The direct gain of OCG over the DG baseline, calculated per seed as $\frac{Z_{\text{OCG}} - Z_{\text{DG}}}{Z_{\text{DG}}} \times 100\%$ and reported as $\mu \pm \sigma$.

Unless otherwise noted, all synthetic results are reported as Mean ± Standard Deviation.

6.1 Real-Data Validation

As the first step of validation, we ran the algorithms on the real-world 2026 Yahoo and FanGraphs projections. As seen in Table 4, OCG successfully trims the point loss caused by naive greedy drafting, confirming its practical viability on real MLB data structures before proceeding to the more comprehensive synthetic validations.

Table 4: Real-data validation (Gap measured in optimal gap ratio versus IP).

Scoring Method	Method	Optimal Gap Ratio
Yahoo 2026	Opportunity Cost Greedy	**0.50%**
	Direct Greedy	3.24%
FanGraphs 2026	Opportunity Cost Greedy	**1.24%**
	Direct Greedy	3.36%

6.2 Factor A: Points Distribution

Table 5 summarizes the impact of the talent distribution. Under the **Baseline (Normal)** distribution, OCG achieves a tight gap of 1.92%, reducing the error of the DG baseline by more than half. The advantage is most pronounced in **Star-Heavy** markets (S3), where OCG prevents elite talent from being snatched by opponents, yielding a **6.10%** average improvement in total roster value.

Table 5: Results for Factor A: Points Distribution. Values represent the optimality gap (%) ± standard deviation.

Level	DG Gap (%)	OCG Gap (%)	Improvement (%)
Normal (Baseline)	$4.79\% \pm 1.18\%$	$\mathbf{1.92}\% \pm \mathbf{0.57}\%$	$3.02\% \pm 1.34\%$
Uniform	$3.24\% \pm 0.77\%$	$\mathbf{1.44}\% \pm \mathbf{0.59}\%$	$1.87\% \pm 0.65\%$
High-Low (Star-Heavy)	$8.39\% \pm 2.40\%$	$\mathbf{2.87}\% \pm \mathbf{1.32}\%$	$\mathbf{6.10}\% \pm \mathbf{3.46}\%$

6.3 Factor B: Position Eligibility Distribution

Table 6 illustrates performance across different positional flexibility environments. Compared to the **Baseline (Roster-Ratio)**, *point-flexible* is the most forgiving environment (gap drops to 1.34%) because top players can seamlessly plug holes in the roster. Conversely, *single-position* creates a rigid, unforgiving draft. Even when mistakes are impossible to patch with utility players, OCG dominates Direct Greedy (0.94% vs 5.11%), proving that looking one pick ahead prevents catastrophic positional deficits later in the draft.

Table 6: Results for Factor B: Position Eligibility Distribution. Values represent the optimality gap (%) ± standard deviation.

Level	DG Gap (%)	OCG Gap (%)	Improvement (%)
Roster-Ratio (Baseline)	$4.79\% \pm 1.18\%$	$\mathbf{1.92}\% \pm \mathbf{0.57}\%$	$3.02\% \pm 1.34\%$
Uniform-by-Type	$4.89\% \pm 0.62\%$	$\mathbf{1.07}\% \pm \mathbf{0.43}\%$	$4.02\% \pm 0.82\%$
Point-Flexible	$5.36\% \pm 1.66\%$	$\mathbf{1.34}\% \pm \mathbf{0.68}\%$	$4.27\% \pm 1.83\%$
Single-Position	$5.11\% \pm 1.48\%$	$\mathbf{0.94}\% \pm \mathbf{0.65}\%$	$\mathbf{4.42}\% \pm \mathbf{1.95}\%$

6.4 Factor C: Market Conditions and ADP Uncertainty

Table 7 demonstrates performance under chaotic draft markets. We split the analysis into two sweeps: varying market noise (σ_{adp}) and varying availability tolerance (δ).

When the market is perfectly efficient ($\sigma_{\text{adp}} = 0$), predicting opponent behavior is trivial, and both algorithms perform exceptionally well. As noise increases to $\sigma_{\text{adp}} = 60$, the draft becomes unpredictable. Direct Greedy struggles significantly (gap over 5%), whereas OCG remains highly robust (2.43%). Similarly, whether the availability tolerance (δ) is incredibly strict (-10) or highly permissive (10), OCG consistently keeps the gap under 2%. Because OCG bases its priority on the *relative difference* (Opportunity Cost) rather than absolute values, it inherently hedges against market volatility.

Table 7: Factor C: Market Conditions and ADP Uncertainty results. The first sweep tests market noise (σ_{adp}) with fixed $\delta = 0$. The second tests nonnegative tolerance (δ) with fixed $\sigma_{\text{adp}} = 30$. Values represent the optimality gap (%) ± standard deviation.

Sweep	Level	DG Gap (%)	OCG Gap (%)	Improvement (%)
Noise	$\sigma_{\text{adp}} = 0$ (Efficient)	$1.47\% \pm 0.48\%$	$\mathbf{0.48\% \pm 0.23\%}$	$1.00\% \pm 0.50\%$
	$\sigma_{\text{adp}} = 30$ (Moderate)	$4.79\% \pm 1.18\%$	$\mathbf{1.92\% \pm 0.57\%}$	$3.02\% \pm 1.34\%$
	$\sigma_{\text{adp}} = 60$ (Chaotic)	$5.56\% \pm 1.58\%$	$\mathbf{2.43\% \pm 1.55\%}$	$\mathbf{3.33\% \pm 1.43\%}$
Tolerance	$\delta = 0$ (Baseline)	$4.79\% \pm 1.18\%$	$\mathbf{1.92\% \pm 0.57\%}$	$3.02\% \pm 1.34\%$
	$\delta = +5$	$5.54\% \pm 1.57\%$	$\mathbf{1.64\% \pm 0.75\%}$	$\mathbf{4.16\% \pm 1.50\%}$
	$\delta = +10$ (Permissive)	$5.42\% \pm 1.21\%$	$\mathbf{1.72\% \pm 0.95\%}$	$3.92\% \pm 1.03\%$

6.5 Factor D: Player Demand Ratio

Table 8 highlights scaling under tight vs. loose supply (Player Demand Ratio D). When supply is brutally tight ($D = 1$, meaning almost every player must be drafted), Direct Greedy collapses (8.81% gap) as it blindly paints itself into a corner. OCG, aware of the impending scarcity, navigates the tight market with only a 2.64% gap. In a high-abundance scenario ($D = 10$), OCG drops the error rate to a mere 1.16%.

Table 8: Factor D: Player Demand Ratio results (Fixed Roster Scale = 1). Values represent the optimality gap (%) ± standard deviation.

Level	DG Gap (%)	OCG Gap (%)	Improvement (%)
D = 3 (Baseline)	$4.79\% \pm 1.18\%$	$\mathbf{1.92\% \pm 0.57\%}$	$3.02\% \pm 1.34\%$
D = 1 (Demand: High)	$8.81\% \pm 2.19\%$	$\mathbf{2.64\% \pm 1.23\%}$	$\mathbf{6.82\% \pm 3.13\%}$
D = 10 (High Abundance)	$3.29\% \pm 0.85\%$	$\mathbf{1.16\% \pm 0.51\%}$	$2.20\% \pm 0.83\%$

6.6 Large-Scale Stress Testing

This section presents the most critical argument for our front office pitch: **Scalability**. While the IP finds perfect rosters in smaller instances, what happens when the scouting department dumps a universe of hundreds of thousands of eligible players into the database?

Table 9 showcases the **Large-Scale Stress Test**. To understand the complexity of the IP, we approximate its search space by the variable count. For n available players, p roster positions, and a roster size of r, the approximate number of decision variables is:

$$\text{Approx. Variables} = n \times (1 + p + r)$$

This formula encompasses the player-selection variables (y_i), player-position assignment variables (x_{ip}), and player-round availability variables (z_{ik}). As we scale the roster size to 256 and the player pool to 184,320, the approximate variable count reaches nearly 50 million.

The Branch-and-Bound tree of the IP explores an exponentially growing space. As a result, the IP runtime explodes from 4 seconds to 276 seconds, and eventually hits our predefined 30-minute (1800 seconds) solver time limit for the largest target, triggering a `TIMEOUT_ERROR`. In stark contrast, confirming our $O(n \log n)$ complexity analysis, **Opportunity Cost Greedy processes the massive 50-million variable equivalent environment in just 23.286 seconds**.

Table 9: Stress Test: Execution Times for DG, OCG, and IP.

Instance Level	Players	Teams	Picks/Team	DG Time(s)	OCG Time(s)	IP Time(s)	IP Status
stress_small	5,760	12	48	0.790	0.852	4.235	OPTIMAL
stress_medium	9,600	15	64	0.951	1.144	12.230	OPTIMAL
stress_large	21,600	15	96	1.981	2.344	54.267	OPTIMAL
stress_xlarge	64,000	20	160	6.172	10.360	276.895	OPTIMAL
timeout_target	184,320	24	256	17.760	23.286	**1869.805**	**TIMEOUT**

Table 10: Stress Test: DG and OCG Optimality Gaps Relative to IP.

Instance Level	DG Gap (%)	OCG Gap (%)
stress_small	2.430%	**1.098%**
stress_medium	1.771%	**0.646%**
stress_large	0.940%	**0.407%**
stress_xlarge	0.270%	**0.137%**
timeout_target	–	–

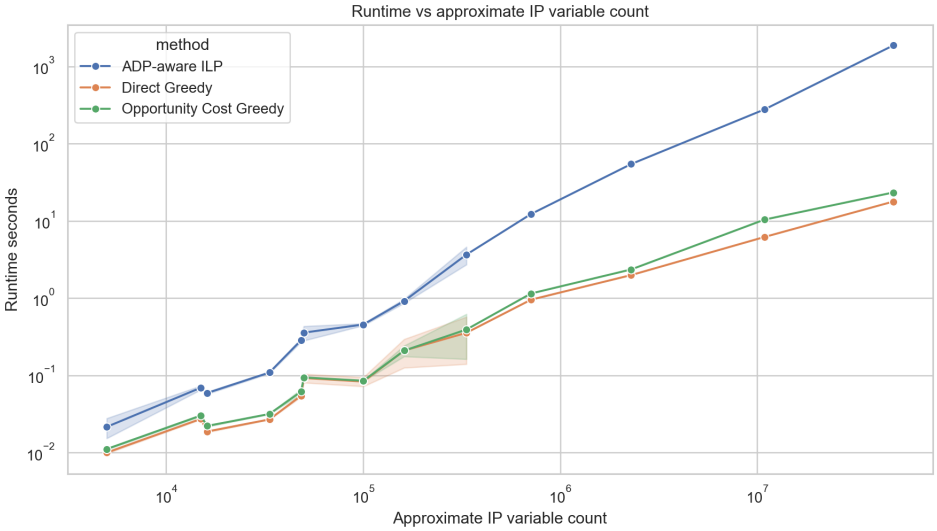

Figure 3: Runtime comparison for Stress Testing. IP scales exponentially, while heuristics scale linearly.

6.7 Strategic Insights and Competitive Edge

After evaluating the algorithm across 12 distinct scenarios, we identify the conditions where the **Opportunity Cost Greedy (OCG)** algorithm provides the most significant strategic advantage. Table 11 highlights the top-performing scenarios based on the improvement over the baseline.

Table 11: Key Strategic Scenarios: Maximum Improvement of OCG over DG.

Factor	**Scenario ID**	**Environment Condition**	**Improvement (%)**
Points Distribution (Factor A)	S3	Points: Star-Heavy	**+6.10%**
Position Eligibility (Factor B)	S6	Pos: Single-Position	**+4.42%**
Market Conditions (Factor C)	S9	Market: Chaotic ($\delta = +5$)	**+4.16%**
Player Demand Ratio (Factor D)	S11	Demand: High ($D = 1$)	**+6.82%**

Analysis of the Competitive Edge:

- **Securing Elite Talent (S3):** In star-heavy drafts, the "cost of waiting" for one more turn is mathematically massive. OCG quantifies this cost, ensuring elite assets are secured before the value "cliff" occurs.
- **Pos: Single-Position (S6):** When positional flexibility is eliminated, early greedy mistakes are impossible to patch. OCG's foresight prevents these catastrophic positional deficits, maintaining high optimality.
- **Market: Chaotic (S9):** Even when opponents' behavior is unpredictable, OCG maintains a robust lead, proving its delay-cost heuristic acts as a natural hedge against uncertain market conditions.
- **Demand: High (S11):** In environments where almost every player is drafted ($D = 1$), DG often suffers from "positional blindness," leading to a 6.82% loss in roster value. OCG's foresight prevents these late-draft collapses.

7 Conclusions and Business Recommendations

For modern sports roster construction, a successful draft can shape the long-term quality of the team. While player evaluations have largely converged across many data-driven environments, draft execution remains a highly exploitable inefficiency.

This project successfully translated the chaotic, interdependent environment of a fantasy snake draft into a rigorous Operations Research model. We demonstrated that relying on naive "best player available" mentalities (Direct Greedy) bleeds significant roster value, especially when the talent pool is top-heavy or positional flexibility is low.

While the exact ADP-aware IP provides flawless rosters, the limitless nature of the baseball player pool renders exact optimization operationally dead-on-arrival due to exponential runtime scaling. Our proposed **Opportunity Cost Greedy (OCG)** algorithm brilliantly bridges this gap. By calculating the delay-cost of waiting just one pick into the future, OCG captures the strategic essence of the IP. It consistently minimizes the optimality gap under severe market stress and processes 50-million variable equivalent draft instances in less than 25 seconds.

Limitations & Future Extensions: ADP is only a proxy for opponent behavior; human managers exhibit biases (e.g., hometown favorites) that ADP misses. Furthermore, point projections carry standard deviations (injury risk, slump risk) not currently modeled. Future iterations of this tool could incorporate probabilistic opponent modeling and Monte Carlo simulations on projection variance, upgrading OCG from a deterministic solver to a fully probabilistic draft-day AI.

Ultimately, this report proves that the OCG algorithm provides a strong, deployable balance between execution speed and solution quality. While our empirical validation was conducted on baseball data, the underlying **cost-of-delay logic** is highly generalizable. It can be extended to any domain involving competitive talent recruitment or procurement scenarios, where multiple agents must secure scarce, heterogeneous assets in a rapidly changing market environment under real-time constraints.